



EST160 - Aula 03

Lucas Sousa

25 de Agosto de 2026

Conteúdos

Importação, Leitura e Escrita de Dados

Inspeção

Introdução

Agora estaremos interessados em

- ▶ Identificar diferentes fontes e formatos de dados
- ▶ Importar dados de arquivos e fontes externas utilizando R
- ▶ Reconhecer e verificar os tipos das variáveis após a importação
- ▶ Compreender coerção de tipos e suas consequências
- ▶ Criar e manipular estruturas tibble
- ▶ Identificar problemas de organização em dados brutos

Introdução

Quando lemos dados, precisamos que a máquina interprete

- ▶ linhas e colunas;
- ▶ separadores (" , " " ; " " \t ");
- ▶ cabeçalhos;
- ▶ valores faltantes
- ▶ números, textos e datas

Para isso, vamos utilizar o pacote readr.

Introdução

Dados Estruturados

Dados Não Estruturados

Dados Semi-Estruturados

Leitura de Dados

Podemos precisar ler dados em diferentes formatos.

- ▶ *Flat files* (ex. CSV, TSV)
- ▶ *Spreadsheets*/planilhas (ex. arquivos excel e google sheets)
- ▶ Banco de Dados (Data Base Interface) (ex. SQLite)
- ▶ APIs (application programming interface)/Web (arquivos JSON)
- ▶ HTML

Leitura de Dados

Muitas vezes precisaremos interpretar e converter valores do nosso banco de dados.

Podemos ter a mesma informação em diferentes formatação, entradas que informam dados faltantes (ex. escrever N/A numa cédula ao invés de deixar a entrada vazia), problemas de organização de tabela em formatados inapropriados.

Pacotes

Nesse etapa, focaremos nos pacotes `readr`, `readxl` e `tibble`.



Lembrando que ao instalar o `tidyverse`, instalamos todos os esses pacotes, mas eles podem ser instalados manualmente.

Arquivos CSV e TSV

Arquivos CSV (comma-separated values) e TSV (tab-separated values) são arquivos de textos com uma estrutura simples e sem formatação.

Utilizamos o termo *flat files* por esses arquivos são sequencias, mas não possuem estruturas complexas, como camadas, relações o metadados.

Geralmente a primeira linha desses arquivos nos indica os nomes das nossas variáveis, e não temos tipos explícitos de variáveis no arquivo.

CSV e TSV

Na verdade, temos um metadado que precisa ser encarado com cuidado em flat files, que é a codificação (ver encoding)

Essa informação pode nos dar alguns problemas de leitura, principalmente por utilizarmos caracteres como ã, ç, õ, etc.

CSV e TSV

Na verdade, temos um metadado que precisa ser encarado com cuidado em flat files, que é a codificação (ver encoding)

Essa informação pode nos dar alguns problemas de leitura, principalmente por utilizarmos caracteres como ã, ç, õ, etc.

Funções:

```
read.csv()
```

```
read.tsv()
```

Outras funções

`read_csv2()`: lê arquivos separados por ponto e vírgula. Arquivos que usam ; invés de , para separar campos, e são comuns em países que usam vírgula como marcador decimal (como Brasil).

`read_delim()`: é uma função mais geral na qual podemos definir qualquer delimitador. Ela tentará deduzir o delimitador se este não for especificado.

`read_fwf()`: lê arquivos com comprimento fixo; Podemos especificar os campos pela sua largura com `fwf_widths()` ou pela sua posição com `fwf_positions()`.

`read_table()`: podemos ter tabelas com linhas de tamanho diferente.

Lendo Dados de Múltiplas Bases

É comum que os dados estejam espalhados em múltiplos arquivos ao invés de um só.

Podemos ter dados de vendas para diferentes meses do ano, como `01-vendas.csv` para janeiro e `02-vendas.csv` para Fevereiro e etc.

Podemos usar a função `read_csv()` para ler todos os dados de uma vez e pô-los em um único banco de dados.

Múltiplas Bases

```
sales_files <- c("data/01-sales.csv", "data/02-sales.csv", "data/03-sales.csv")
```

```
read_csv(sales_files, id = "file")
```

```
#> # A tibble: 19 x 6
```

#>	file	month	year	brand	item	n
#>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
#> 1	data/01-sales.csv	January	2019	1	1234	3
#> 2	data/01-sales.csv	January	2019	1	8721	9
#> 3	data/01-sales.csv	January	2019	1	1822	2
#> 4	data/01-sales.csv	January	2019	2	3333	1
#> 5	data/01-sales.csv	January	2019	2	2156	9
#> 6	data/01-sales.csv	January	2019	2	3987	6
#> #	13 more rows					

Múltiplas bases

Precisamos notar que o código anterior vai funcionar se os dados estiverem em uma pasta de nosso projeto. Também podemos baixar arquivos por meio de links.

Por exemplo, os dados anteriores podem ser baixados da seguinte forma:

```
sales_files <- c(
  "https://pos.it/r4ds-01-sales",
  "https://pos.it/r4ds-02-sales",
  "https://pos.it/r4ds-03-sales"
)
```

```
read_csv(sales_files, id = "file")
```

O argumento `id` adiciona uma nova coluna identificando de onde veio cada dado. Isso é útil quando não há outras formas de descobrir de onde os dados vieram.

Escrevendo uma base manualmente

Também podemos escrever uma base de dados manualmente, se for necessário.

```
tibble(  
  x = c(1, 2, 5),  
  y = c("h", "m", "g"),  
  z = c(0.08, 0.83, 0.60)  
)  
#> # A tibble: 3 x 3  
#>       x       y       z  
#>   <dbl> <chr> <dbl>  
#> 1     1     h    0.08  
#> 2     2     m    0.83  
#> 3     5     g    0.6
```

Um uso que pode ser mais comum para criar base de dados manualmente talvez seja o uso de loops. Por exemplo, atribuindo vetores a cada linha. **Mas temos funções para fazer isso.**

Escrevendo manualmente

Note que os vetores que escrevemos anteriormente nos dão as colunas da nossa base.

Algumas vezes isso pode ser confuso. Se quisermos uma alternativa que atribua linhas, podemos utilizar `tribble()` (abreviação de *transposed tibble*), que deixa a gente atribuir os dados linha por linha.

Nesse caso, os títulos das colunas começam com `~` e as entradas são separadas por vírgula.

Escrevendo manualmente

```
tribble(  
  ~x, ~y, ~z,  
  1, "h", 0.08,  
  2, "m", 0.83,  
  5, "g", 0.60  
)  
#> # A tibble: 3 x 3  
#>       x       y       z  
#>   <dbl> <chr> <dbl>  
#> 1     1     h    0.08  
#> 2     2     m    0.83  
#> 3     5     g    0.6
```

Mas nesse caso não podemos atribuir vetores às linhas.

Arquivos TSV

O parâmetro que utilizamos para tab character é `\t`, enquanto `\n` representa quebra de linhas

Na prática não vemos essas divisões, já que são espaços e quebras de linha

```
dados <- read.tsv("dados.tsv")
```

ou

```
dados <- read.delim("dados.tsv", delim = "\t")
```

Arquivos TSV

A principal vantagem é quando os dados contêm vírgulas e não queremos que sejam interpretadas como divisão de células.

Por exemplo, podemos ter o um endereço como entrada. Para o CSV, precisaríamos escrever os valores entre aspas

```
Name,Address
```

```
John,"123 Main St, New York"
```

Arquivos TSV

A principal vantagem é quando os dados contêm vírgulas e não queremos que sejam interpretadas como divisão de células.

Por exemplo, podemos ter o um endereço como entrada. Para o CSV, precisaríamos escrever os valores entre aspas

```
Name , Address  
John , "123 Main St , New York"
```

Para TSV, podemos escrever

```
Name      Address  
John      123 Main St , New York
```

Planilhas

Outra categoria de arquivos bastante comuns são as *planilhas*.

Os formatos mais populares são *excel* e *google sheets*.

Nesse caso, teremos mais informações, como abas, além de podermos ter tipos de variáveis em nossos dados.

Planilhas

Outra categoria de arquivos bastante comuns são as *planilhas*.

Os formatos mais populares são *excel* e *google sheets*.

Nesse caso, teremos mais informações, como abas, além de podermos ter tipos de variáveis em nossos dados.

Aqui podemos ter outras questões quanto a organização da planilha, como cabeçalhos, células mescladas, linhas em branco, unidades variadas, representações de dados faltantes (–, N/A, ?) ao invés de células vazias, formatação com significados (ex. cores indicando atributos)

Arquivos Excel

Para lidar com as planilhas do Excel utilizamos o pacote `readxl`, que é instalado junto com o tidyverse, mas precisa ser chamado separadamente.

Também temos o pacote `writexl` para criar planilhas do excel.

Arquivos Excel

No pacote `readxl` temos diferentes funções para ler os arquivos do Excel.

- ▶ `read_xls()` lê arquivos em formato XLS
- ▶ `read_xlsx()` lê arquivos em formatos XLSX
- ▶ `read_excel()` lê os dois formatos e tenta adivinhar o tipo baseado no arquivo.

Essas funções tem uma sintaxe similar às funções que exploramos anteriormente.

Arquivos Excel

Lembrem que planilhas podem ter diferentes abas (*worksheets*)

Podemos ler uma única aba da planilha com um argumento `sheet` na função `read_excel()`. O padrão é a primeira aba.

```
read_excel("data/penguins.xlsx", sheet = "Torgersen Island")
```

Tipos de Dados no Excel

Quando lidamos com arquivos CSV, todos os valores são *strings*, embora os dados não sejam necessariamente *strings*.

Isso significa que o CSV não contém nenhuma informação sobre o tipo de cada variável (se é lógica, número, string, etc.). Nesse caso, o `readr` vai tentar adivinhar o tipo. Falaremos mais sobre isso posteriormente.

Tipos de Dados no Excel

Para as planilhas do Excel, os dados são mais complexos. Uma célula pode ser uma das quatro coisas a seguir:

- ▶ Valores Booleanos (TRUE, FALSE ou NA)
- ▶ Um número
- ▶ Uma data, que pode incluir coisas do tipo "24/8/26" ou "24/8/26 3:00 PM"
- ▶ Um texto string

Arquivos Excel

Um argumento importante para `read_excel()` é `na`, visto que podemos precisar estabelecer os padrões para os que são NAs.

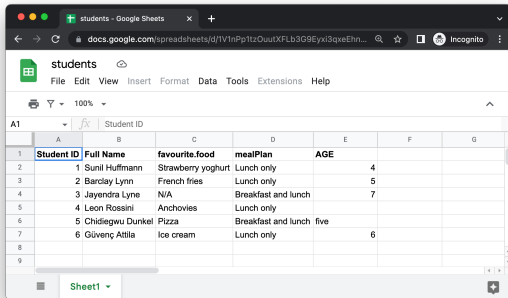
```
read_excel("dados.xlsx", na = c("-", "N/A", "?"))
```

Podemos escrever dados no formato Excel com a função `write_excel()`

```
write_excel(dados_limpos, path = "data/dados-  
limpos.xlsx")
```

Google Sheets

O *Google sheets* (Google planilhas) é outro ambiente comum para trabalharmos com planilhas.



The screenshot shows a Google Sheets interface with a spreadsheet titled 'students'. The spreadsheet has a table with 7 columns: Student ID, Full Name, favourite.food, mealPlan, AGE, and two empty columns. The data is as follows:

	A	B	C	D	E	F	G
1	Student ID	Full Name	favourite.food	mealPlan	AGE		
2	1	Sunil Huffmann	Strawberry yoghurt	Lunch only	4		
3	2	Barclay Lynn	French fries	Lunch only	5		
4	3	Jayendra Lyne	N/A	Breakfast and lunch	7		
5	4	Leon Rossini	Anchovies	Lunch only			
6	5	Chidiagwu Dunkel	Pizza	Breakfast and lunch	five		
7	6	Güvenç Atilla	Ice cream	Lunch only	6		
8							
9							

Podemos baixar os arquivos em diferentes formatos, incluindo .xlsx e .csv, mas também podemos receber os dados diretamente de links das planilhas com o pacote googlesheets4. O pacote deve ser chamado explicitamente.

Google Sheets

O primeiro argumento que usamos para a função `read_sheets` é o URL do arquivo. Isso nos retornará uma *tibble*.

```
students_url <- "https://docs.google.com/spreadsheets/d/1
V1nPp1tzOuutXFLb3G9Eyxi3qxeEhnOXUzL5_BcCQ0w"

students <- read_sheet(students_url)
```

Podemos atribuir o link a uma variável, para que o código fique menos poluído.

Google Sheets

Assim como em `read_excel()`, podemos fornecer os nomes das colunas, informar quais são os NAs e o tipo de coluna com a função `read_sheet()`.

```
students <- read_sheet(  
  students_sheet_id ,  
  col_names = c("student_id", "full_name", "favourite_food", "  
    meal_plan", "age"),  
  skip = 1,  
  na = c("", "N/A"),  
  col_types = "dcccc"  
)
```

Google Sheets

Assim como em `read_excel()`, podemos fornecer os nomes das colunas, informar quais são os NAs e o tipo de coluna com a função `read_sheet()`.

```
students <- read_sheet(  
  students_sheet_id ,  
  col_names = c("student_id", "full_name", "favourite_food", "  
    meal_plan", "age"),  
  skip = 1,  
  na = c("", "N/A"),  
  col_types = "dcccc"  
)
```

Note que os tipos de variáveis nas colunas é definido usando siglas. “dcccc” significa “double, char, char, char, char.”

O argumento `skip` pula **linhas** nesse caso. Se usarmos um número maior, pulares até aquela linha. Para pular colunas, precisamos determinas a região.

Database Interfaces (DBI)

DBIs são interfaces que se conectam a base de dados e executam SQL (Structured Query Language)

SQL (em português linguagem de consulta estruturada) é uma linguagem de domínio específico desenvolvida para gerenciar dados relacionais em um sistema de gerenciamento de banco de dados, ou para processamento de fluxo de dados em um sistema de gerenciamento de fluxo de dados.

```
library(DBI) # conecta a base de dados
```

```
library(dbplyr) # traduz o código para SQL
```

Pacote Arrow

Para grandes colunas de dados, temos o pacote arrow.

Esse pacote permite trabalhar com arquivos no formato Parquet, que armazena os dados numa estrutura binária (ver documentação).

Pacote Arrow

Para grandes colunas de dados, temos o pacote arrow.

Esse pacote permite trabalhar com arquivos no formato Parquet, que armazena os dados numa estrutura binária (ver documentação).

Segundo Wickham, Grolemund, *et al.* (2017)

- ▶ *Lento*: manipular base de dados CSV grandes com readr
- ▶ *Rápido*: manipular base de dados CSV grandes com arrow
- ▶ *Muito Mais Rápido*: manipular base de dados parquet com arrow

Dados Hierárquicos

Dados hierárquicos possuem uma estrutura de camadas, onde há uma relação ordenação multidimensional nos dados

Nosso objetivo vai ser transformar dados que estão numa estrutura complexa ou irregular em dados retangulares (estrutura tabular)

Um dos formatos mais comuns de dados hierárquicos é o formato JSON (**J**ava**S**cript **O**bject **N**otation)

Listas

Até então lidamos com data frames que contêm vetores simples, como inteiros, characters, fatores, etc.

Esses vetores são simples porque são homogêneos: cada elemento possui o mesmo tipo de dado.

Se quisermos armazenar elementos de tipos diferentes no mesmo vetor, precisaremos de uma **lista**, criada com a função `list()`

Listas

Criamos listas com diferentes tipos de variáveis

```
x1 <- list(1:4, "a", TRUE)
```

```
x1
```

```
#> [[1]]
```

```
#> [1] 1 2 3 4
```

```
#>
```

```
#> [[2]]
```

```
#> [1] "a"
```

```
#>
```

```
#> [[3]]
```

```
#> [1] TRUE
```

Listas

Podemos nomear os elementos

```
x2 <- list(a = 1:2, b = 1:3, c = 1:4)
```

```
x2
```

```
#> $a
```

```
#> [1] 1 2
```

```
#>
```

```
#> $b
```

```
#> [1] 1 2 3
```

```
#>
```

```
#> $c
```

```
#> [1] 1 2 3 4
```

Listas

E podemos usar `str()` para uma visualização mais simples.

```
str(x1)
```

```
#> List of 3  
#> $ : int [1:4] 1 2 3 4  
#> $ : chr "a"  
#> $ : logi TRUE
```

```
str(x2)
```

```
#> List of 3  
#> $ a: int [1:2] 1 2  
#> $ b: int [1:3] 1 2 3  
#> $ c: int [1:4] 1 2 3 4
```

Listas

Listas podem conter qualquer tipo de elemento, incluindo outras listas

```
x3 <- list(list(1, 2), list(3, 4))  
str(x3)
```

```
#> List of 2  
#> $ :List of 2  
#> ..$ : num 1  
#> ..$ : num 2  
#> $ :List of 2  
#> ..$ : num 3  
#> ..$ : num 4
```

Isso torna esses elementos interessantes para lidar com dados hierárquicos.

Listas

Conforme vão ficando mais complexas, `str()` fica mais útil na visualização da hierarquia das listas.

```
x5 <- list(1, list(2, list(3, list(4, list(5))))  
)  
str(x5)  
#> List of 2  
#> $ : num 1  
#> $ :List of 2  
#> ..$ : num 2  
#> ..$ :List of 2  
#> ...$ : num 3  
#> ...$ :List of 2  
#> ....$ : num 4  
#> ....$ :List of 1  
#> .....$ : num 5
```

Também podemos usar `View()` para uma visão mais dinâmica.

Listas

Também podemos por listas em data frames ou tibbles como colunas (lista-coluna). Isso cria um ecossistema flexível.

```
df <- tibble(  
  x = 1:2,  
  y = c("a", "b"),  
  z = list(list(1, 2), list(3, 4, 5))  
)  
df
```

```
#> # A tibble: 2 x 3  
#>       x       y       z  
#>   <int> <chr>   <list>  
#> 1     1     a   <list [2]>  
#> 2     2     b   <list [3]>
```

Também podemos separar listas, mas vamos explorar isso quando for necessário.

JSON

JSON é um exemplo comum de formato com dados hierárquicos e pode ser importantes sabermos como se relacionado com o R.

O formato JSON possui seis tipos de dados principais

- ▶ null (que seria o equivalente a NA no R) representa a ausência de dados
- ▶ string
- ▶ número (inteiro, decimal, e notação científica), mas não suporta Inf, -Inf ou NaN
- ▶ Booleanos (true e false em minúsculo)
- ▶ Objetos (equivalente a listas nomeadas no R)
- ▶ Arrays (equivalente a listas não nomeadas no R)

Ao invés de nomes, utilizamos o termo "keys" para objetos. Não há representação nativa de datas.

Vejam Também

- ▶ Formatos Web (HTML)
- ▶ Aprofundamento nos temas

Vejam Wickham, Grolemund, *et al.* (2017) e outros materiais para aprofundar os temas que não discutimos.

CONTINUAMOS NA PRÓXIMA AULA